

УСЛОВНЫЕ КОНСТРУКЦИИ IF-ELIF- ELSE.



Условные конструкции

Условные конструкции - это способ заставить программу делать разные вещи в зависимости от того, что происходит или что введено. Например, если вам нужно, чтобы программа выводила одно сообщение, если введено правильное число, и другое — если оно неправильное.

Ключевая структура: **if-else**

Синтаксис конструкции if-else

```
if условие:  
    # Если условие истинно, выполняется этот блок  
    действие_1  
else:  
    # Если условие ложно, выполняется этот блок  
    действие_2
```

Условные конструкции

Пример

Напишем программу, которая проверяет, является ли число положительным или отрицательным:

```
number = int(input("Введите число: ")) # Вводим число

if number > 0: # Проверяем, больше ли число нуля
    print("Число положительное.")
else: # Если условие не выполняется (то есть число меньше или равно нулю)
    print("Число отрицательное или равно нулю.")
```

Как «думает» компьютер

- Программа начинает с проверки, истинно ли условие «`number > 0`». Если да, выполняется блок с командой `print`(«Число положительное.»).
- Если условие ложно, выполняется блок после `else`, и на экран выводится сообщение «Число отрицательное или равно нулю».

Условное выражение

Условное выражение – это логическое утверждение, которое может быть истинным или ложным. Оно может содержать операции сравнения.

Операции сравнения:

- **>** (больше)
- **<** (меньше)
- **>=** (больше или равно)
- **<=** (меньше или равно)
- **==** (равно)
- **!=** (не равно)

Условное выражение

Пример:

```
if 5 > 3:  
    print("5 больше 3")
```

Задание №1:

Напишите программу которая будет сравнивать два числа и выводить то, которое меньше

Задание №2:

Напишите программу которая будет получать от пользователя два числа и выводить на экран то, которое больше.

Вложенные условия

Иногда можно вкладывать одну условную конструкцию внутрь другой, чтобы делать более сложные проверки. Это называется **вложенными условиями**.

Пример:

```
age = 16
genre = "ужасы"

if age >= 18:
    if genre == "ужасы":
        print("Советуем посмотреть фильм 'Заклятие'.")
    else:
        print("Я знаю только фильмы ужасов")
else:
    print("Ты ещё слишком молод, чтобы смотреть ужасы.")
```

Условие «if» без «else»

Иногда можно использовать только if, без else, если мы хотим выполнить действие, только если условие истинно, а если оно ложно — ничего не делать.

Пример:

```
number = int(input("Введите число: ")) # Вводим число

if number % 2 == 0: # Если число чётное
    print("Число чётное.")
```

Здесь программа только проверяет, чётное ли число. Если да — выводит сообщение, если нет — программа ничего не выводит.

Использование логических операторов

Логические операторы:

- **and** — логическое «И»: условие истинно, если истинны оба выражения.
- **or** — логическое «ИЛИ»: условие истинно, если истинно хотя бы одно выражение.
- **not** — логическое «НЕ»: инвертирует результат (истинное становится ложным и наоборот).

Пример **and**

```
temperature = 30
if temperature > 20 and temperature < 35:
    print("Погода идеальна для прогулки!")
```

Конструкция if проверяет — находится ли переменная temperature в промежутке боль 20 и меньше 30.

Использование логических операторов

Пример **or**

```
name = "Василиса"
if name == "Василиса" or name == "Василий":
    print("Привет, Вася! :)")
```

Программа выполнит действие внутри конструкции if при любом из двух имён.

Пример **not**

```
is_raining = False
if not is_raining:
    print("Сегодня можно идти гулять без зонта!")
```

Программа проверяет значение переменной is_raining. Переменная, которая изначально имеет значение False, внутри условной конструкции принимает значение True.

Особенности синтаксиса

- **Отступы:**

Очень важно правильно оформлять **отступы**. В Python отступы определяют, какой код принадлежит к какому блоку. Обычно используется 4 пробела для каждого уровня вложенности.

- **Двоеточие:**

После ключевого слова if, elif или else **ВСЕГДА** ставится двоеточие **:**, чтобы Python понял, что начинается блок кода.

- **Форматирование:**

Крайне рекомендуется писать код чисто и понятно, чтобы было легче его читать и поддерживать.

Расширенная структура if-elif-else

Когда мы хотим проверить не одно, а **несколько** возможных условий, мы используем конструкцию **if — elif — else**.

if — если

elif — иначе если

else — иначе

```
if условие1:
    # блок кода 1 (выполняется, если условие1 – True)
elif условие2:
    # блок кода 2 (если условие1 – False, но условие2 – True)
elif условие3:
    # блок кода 3 (если первые два – False, а это – True)
...
else:
    # блок кода по умолчанию (если ни одно из условий не подошло)
```

Расширенная структура if-elif-else

if – если

elif – иначе если

else – иначе

```
score = int(input("Сколько баллов ты набрал? "))
if score >= 90:
    print("Оценка: 5 (Отлично)")
elif score >= 70:
    print("Оценка: 4 (Хорошо)")
elif score >= 50:
    print("Оценка: 3 (Удовлетворительно)")
else:
    print("Оценка: 2 (Неудовлетворительно)")
```

Важно понимать

- **if** проверяется **всегда**.
- **elif** проверяется только если **if** — **False**.
- **else** — это запасной вариант, он срабатывает, если **всё остальное не подошло**.
- Старайтесь **не дублировать условия** — Python сам проверит по порядку.
- **elif** может быть **сколько угодно**, а **else** — только **один** и в самом конце.
- **Без else тоже можно** — просто программа ничего не сделает, если ни одно условие не подошло.

Когда использовать if-elif-else?

- Когда у вас **несколько взаимоисключающих условий** (т.е. только одно из них должно быть выполнено).
- Когда нужно **проверить значения переменной на несколько вариантов** (например, числа, время, текст и т.д.).
- Когда вы хотите **сделать код читаемым и понятным**, не используя вложенные условные конструкции.

ЦИКЛ FOR.



Что такое цикл?

Цикл – это специальная конструкция, которая позволяет выполнять действия несколько раз автоматически.

Один из самых популярных типов циклов в Python — это цикл **for**. Он используется, когда мы знаем, **сколько раз нужно повторить действие**, или хотим пройтись по набору элементов.

Простая структура цикла **for**:

```
for переменная in последовательность:  
    действия внутри цикла
```


Цикл for

```
for i in range(1, 6):  
    print(i)
```

Что здесь происходит:

range(1, 6) — это последовательность чисел от 1 до 5 (6 не входит!),

i — это переменная, которая по очереди принимает значения из **range**,

print(i) — выводим текущее значение **i**.

Цикл for

range() — это функция, которая создаёт последовательность чисел. Мы можем использовать эту последовательность, чтобы выполнять команды в цикле for много раз, каждый раз с новым числом.

Синтаксис:

range(СТАРТ, СТОП, ШАГ)

«СТАРТ» — с какого числа начать (по умолчанию 0)

«СТОП» — до какого числа идти (не включая его!)

«ШАГ» — на сколько увеличивать (или уменьшать) каждое следующее число (по умолчанию +1)

Цикл for

Каждое число, которое создаёт **range()**, временно сохраняется в переменной (часто её называют **i** или **num**), и цикл **for** повторяет команды для каждого такого числа.

```
for i in range(3):  
    print("Привет!", i)
```

Почему СТОП не включается?

Так устроен Python — это делается для удобства и точности. Если ты хочешь 10 повторений, ты просто пишешь **range(10)** — и получаешь числа от 0 до 9 (всего 10 штук).

В любом языке программирования отчет начинается с 0

Переменные в цикле

Функция **range()** не обязательно должна содержать только числа. Мы можем передавать **переменные**, которые содержат нужные значения.

```
start = 1
stop = 5
for i in range(start, stop):
    print(i)
```

Использование переменных делает программу **более гибкой**:

- Мы можем заранее спросить у пользователя, сколько раз повторять цикл.
- Мы можем менять параметры range() без изменения самого цикла.

```
count = int(input("Сколько раз вывести слово 'Привет'? "))

for i in range(count):
    print("Привет!", i)
```

Спасибо за внимание!

